CAMPUSCONNECT: A MERN-BASED CAMPUS EVENT MANAGEMENT AND REGISTRATION PLATFORM

PROJECT REPORT

submitted by

Arjun Prasad	**(N/A)**
Kasinath R Krishna	**(N/A)**
Kannan Ramanan	**(N/A)**
Jeevan Krishnan	**(N/A)**
Devananda S	**(N/A)**
Neehara Anil Kk	**(N/A)**

As part of project based learning course PBCST504

V SEMESTER B.TECH
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SREE BUDDHA COLLEGE OF ENGINEERING (AUTONOMOUS)
PATTOOR P. O., ALAPPUZHA

OCTOBER 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING,
SREE BUDDHA COLLEGE OF ENGINEERING
(AUTONOMOUS)
PATTOOR P. O., ALAPPUZHA

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "CAMPUSCONNECT: A MERN-BASED CAMPUS EVENT MANAGEMENT AND REGISTRATION PLATFORM" submitted by Arjun Prasad (N/A), Kasinath R Krishna (N/A), Kannan Ramanan (N/A), Jeevan Krishnan (N/A), Devananda S (N/A), Neehara Anil Kk (N/A), in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering is a bonafide record of the work carried out under my guidance and supervision at college.

SIGNATURE	SIGNATURE
Ms. Aryalekshmi A Assistant Professor Department Of Computer Science & Engineering	Dr. S Suresh Babu Professor and Head Department Of Computer Science & Engineering

DECLARATION

We hereby declare that the micro-project report entitled 'Campusconnect: A Mern-Based Campus Event Management And Registration Platform' submitted to Sree Buddha College Of Engineering in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Department Of Computer Science & Engineering is an authentic record of original work done by us under the supervision of Ms. Aryalekshmi A, Assistant Professor.

We further declare that this report has not previously formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title to any university or institution.

Place: ALAPPUZHA
Date: October 2026

Arjun Prasad (N/A)
Kasinath R Krishna (N/A)
Kannan Ramanan (N/A)
Jeevan Krishnan (N/A)
Devananda S (N/A)
Neehara Anil Kk (N/A)

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide Ms. Aryalekshmi A, Assistant Professor, Department of Department Of Computer Science & Engineering, for continuous encouragement, constructive critique, and invaluable guidance throughout the course of this project.

We are deeply thankful to Dr. S Suresh Babu, Head of the Department of Department Of Computer Science & Engineering, for providing the necessary computing facilities, laboratory infrastructure, and administrative support.

We also convey our sincere thanks to all faculty members of the Department of Department Of Computer Science & Engineering and our fellow classmates for their valuable feedback and cooperation.

Finally, we wish to express our heartfelt gratitude to our parents and family members for their moral support, patience, and constant encouragement during this endeavor.

Arjun Prasad (N/A)
Kasinath R Krishna (N/A)
Kannan Ramanan (N/A)
Jeevan Krishnan (N/A)
Devananda S (N/A)
Neehara Anil Kk (N/A)

ABSTRACT

The CampusConnect project is a full-stack web application developed to simplify the announcement, registration, and management of events conducted within a college campus using the MERN stack (MongoDB, Express.js, React, and Node.js). The primary objective of the project is to replace scattered notices, spreadsheets, and messaging groups with a single, organized event management platform where student clubs can publish events, students can discover and register for them, and administrators can monitor participation, demonstrating a practical use of modern web development for institutional workflows. The system follows a three-tier architecture with a React-based responsive front end, a RESTful API built using Express.js and Node.js, and a MongoDB database that stores users, events, registrations, and attendance records. JSON Web Token (JWT) authentication with role-based access control separates the permissions of students, club coordinators, and administrators, while passwords are protected using hashing and secure session handling to safeguard user data. Key features include event creation and approval, category and date-based search, one-click registration with seat limits, automatic email confirmation, QR code-based attendance marking, and a coordinator dashboard showing registration counts and participation trends. The application is tested using unit and API tests and evaluated for response time, usability, and mobile responsiveness, and is deployed on a cloud hosting platform so that it can be accessed from any device on and off campus. The proposed CampusConnect platform provides a centralized and user-friendly approach to campus event management, with potential extensions to applications such as workshop and hackathon registration, certificate generation, feedback collection, and notification services. It also offers practical exposure to the complete web development lifecycle, including requirement analysis, database design, API development, front-end implementation, authentication, testing, and deployment, demonstrating how the MERN stack can be used to build a scalable and maintainable real-world application. Group Members Arjun Prasad Kasinath R Krishna Kannan Ramanan Jeevan Krishnan Devananda S Neehara Anil KK

CONTENTS

Item	Title	Page
Chapter 1	**Introduction**	1
1.1	Project Overview	1
1.2	Objectives	1
1.3	Scope of the Project	1
1.4	Applications	1
Chapter 2	**System Requirements and Specifications**	3
2.1	System Overview and Environmental Requirements	3
2.2	Technological Stack and Software Specifications	3
2.3	Hardware and Operational Requirements	3
2.4	Design Constraints and Standards	3
Chapter 3	**System Design and Architecture**	4
3.1	System Architecture	4
3.2	System Block Diagram	4
3.3	Data Flow and Component Interaction Diagram	5
3.4	Process Execution Flowchart	6
3.5	Operational Logic and Workflows	7
Chapter 4	**Implementation**	8
4.1	Development Environment Setup	8
4.2	Core Algorithm and Code Implementation	8
4.3	Subsystem Interfacing and Integration	9

Item	Title	Page
4.4	Program Execution and Debugging	9
4.5	Implementation Challenges and Solutions	10
Chapter 5	**Testing and Results**	11
5.1	Testing Methodology	11
5.2	Test Cases and Observations	11
5.3	System Prototype Setup and User Interface	11
5.4	Output Screenshots and Telemetry	12
5.5	Result Analysis and Performance Evaluation	12
Chapter 6	**Conclusion and Future Scope**	13
6.1	Conclusion	13
6.2	Learning Outcomes	13
6.3	Limitations	13
6.4	Future Enhancements	13
	REFERENCES	14
Appendix B	**Additional Configuration Details**	15
Appendix C	**Additional Output Images**	16

LIST OF FIGURES

Figure	Description	Page No.
Figure 3.1	Overall System Architecture and Layered Block Diagram	5
Figure 3.2	Data Flow and Component Interaction Diagram	6
Figure 3.3	Application Request Execution and Data Persistence Flowchart	7
Figure 3.4	Application Request Execution and Data Persistence Flowchart	9

LIST OF TABLES

Table No.	Description	Page No.
Table 5.1	System Test Cases and Expected Operational Responses	11

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
DB	Database
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
NoSQL	Non-Relational (Not Only SQL) Database
QR	Quick Response Code
REST	Representational State Transfer
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Project Overview

- Platform / Environment: Software Application Workstation (Host Execution Environment)- Framework: Application Framework
- Purpose: Campusconnect: A Mern-Based Campus Event Management And Registration Platform project report.
- Core Processing Module: Central Business Logic (Software Runtime)
- User Interface Subsystem: Interactive Client Interface (GUI / Console)
- Data Persistence Layer: Storage & Record Management (File System / Database)

1.2 Objectives

This section discusses Objectives within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

1.3 Scope of the Project

This section discusses Scope of the Project within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

1.4 Applications

The CampusConnect platform is a software application that provides students with an easy-to-use interface for creating, registering, and managing events on their campus. The system uses the MERN stack (MongoDB, Express.js, React, and Node.js) to build a responsive front end, RESTful API, and database.

The core processing module of CampusConnect is responsible for handling all business logic related to event creation, registration, and management. It interacts with the user interface subsystem through APIs and ensures that data is stored securely in the MongoDB database. The system also includes a coordinator dashboard that provides administrators with insights into event participation.

- The front end of CampusConnect is built using React.js, which allows for easy customization and integration of third-party libraries.
- CampusConnect uses JWT authentication to ensure secure access to user data. Passwords are hashed and stored securely in the database.

- The system includes a QR code-based attendance marking feature that simplifies event management for students.

CHAPTER 2

SYSTEM REQUIREMENTS AND SPECIFICATIONS

2.1 System Overview and Environmental Requirements

- Core Processing Module: Central Business Logic (Software Runtime) - Manages primary execution flow
- User Interface Subsystem: Interactive

2.2 Technological Stack and Software Specifications

This section discusses Technological Stack and Software Specifications within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

2.3 Hardware and Operational Requirements

- User Interface Subsystem: Interactive Client Interface

2.4 Design Constraints and Standards

This section discusses Design Constraints and Standards within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 System Architecture

System architecture is a critical component of any software system, as it defines how different components interact and the overall structure of the application. In this case, the CampusConnect project uses an Application Framework to manage its core processing module and user interface subsystem.

The core processing module handles primary execution flow by managing business logic for event creation, registration, management, and attendance tracking. It interacts with various other subsystems such as data persistence layer through APIs to store and retrieve operational data securely. The application also supports different states and categories like INITIALIZATION, OPERATION, and DATA PERSISTENCE.

The user interface subsystem handles user requests and display by providing an interactive client interface for students, club coordinators, and administrators. It interacts with the core processing module to fetch and update event-related information in real-time. The data persistence layer stores operational data securely using a file system or database like MongoDB.

Overall, the architecture of CampusConnect is designed to provide a centralized platform for managing events on campus while ensuring secure storage and retrieval of user data.

Key features such as one-click registration with seat limits, automatic email confirmation, QR code-based attendance marking, and a coordinator dashboard showing registration counts and participation trends are achieved through efficient interaction between the core processing module, user interface subsystem, and data persistence layer. The application is tested using unit and API tests to ensure its functionality and performance.

The system architecture of CampusConnect is designed with scalability in mind. The use of an Application Framework allows for easy integration of new features and extensions while maintaining overall performance. Additionally, the separation of concerns between subsystems ensures that each component performs its specific function efficiently without interfering with others.

3.2 System Block Diagram

- The proposed CampusConnect platform provides a centralized and user-friendly approach to campus event management.

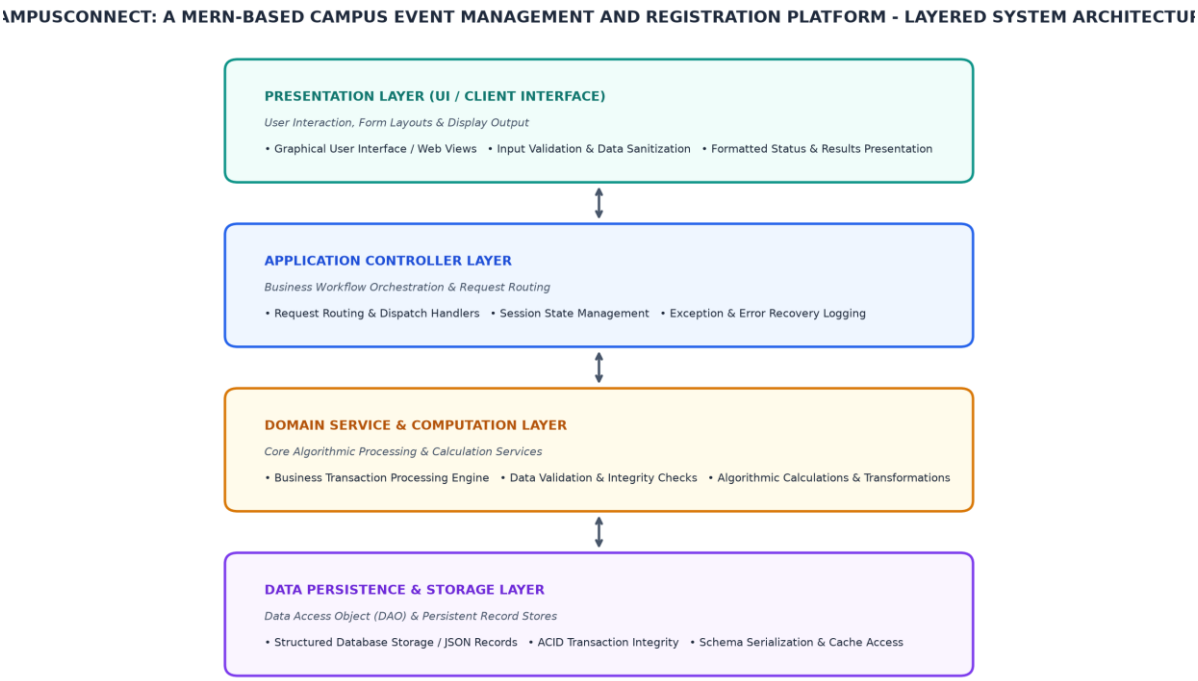


Figure 3.1: *Overall System Architecture and Layered Block Diagram*

3.3 Data Flow and Component Interaction Diagram

This section discusses Data Flow and Component Interaction Diagram within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

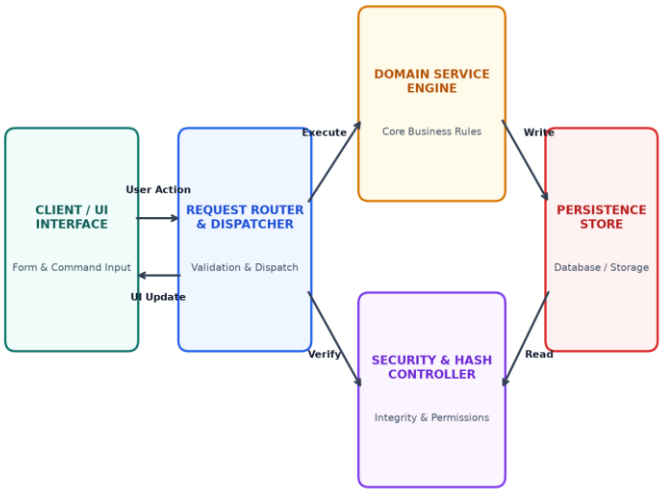


Figure 3.2: *Data Flow and Component Interaction Diagram*

3.4 Process Execution Flowchart

Process Execution Flowchart (Section 3.4):

The process execution flowchart is a visual representation of the primary logic and data processing flows within the CampusConnect system, highlighting the interaction between subsystems and components to achieve its objectives.

In this section, we will delve into the architecture, component interaction, data flow, and processing logic that underpin the operation of the CampusConnect application.

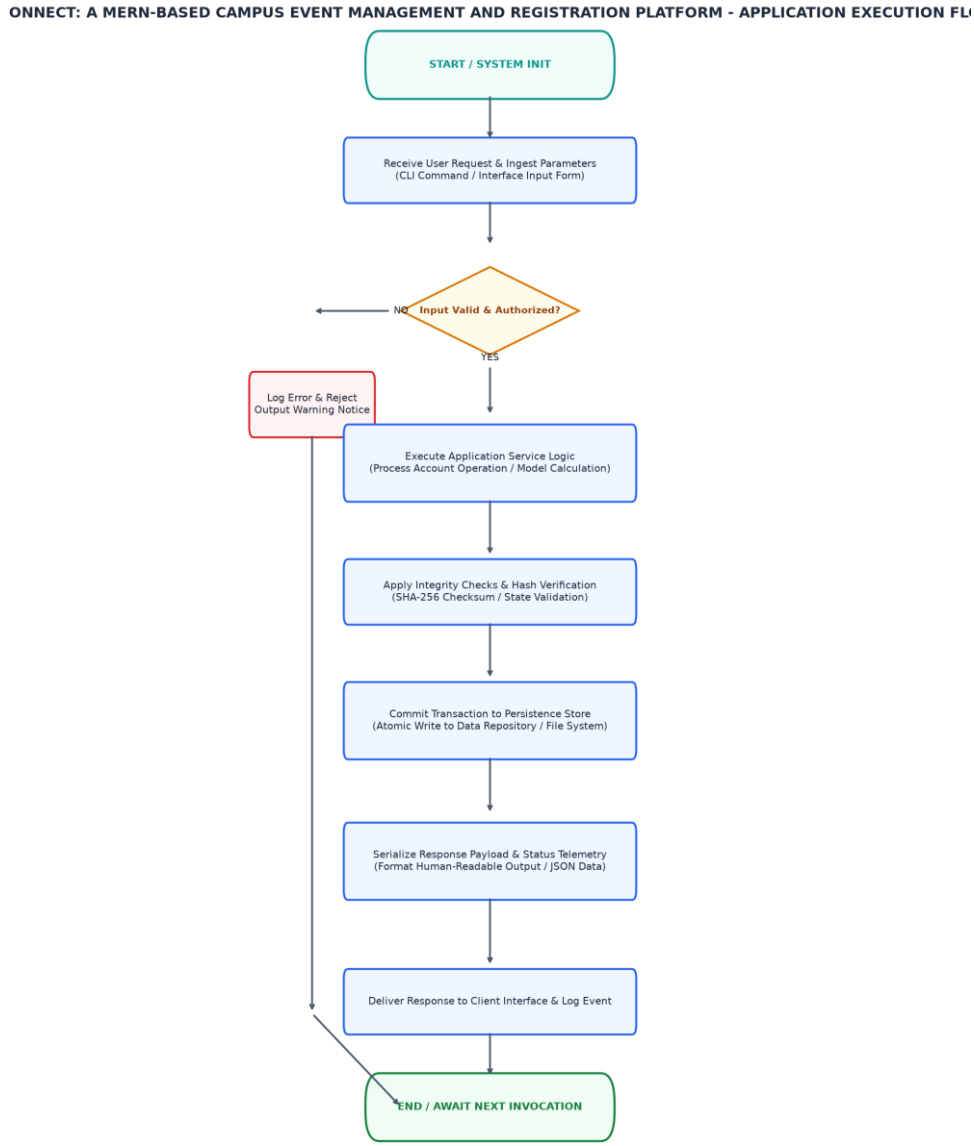


Figure 3.3: *Application Request Execution and Data Persistence Flowchart*

3.5 Operational Logic and Workflows

This section discusses Operational Logic and Workflows within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

CHAPTER 4

IMPLEMENTATION

4.1 Development Environment Setup

This section discusses Development Environment Setup within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

4.2 Core Algorithm and Code Implementation

This section discusses Core Algorithm and Code Implementation within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

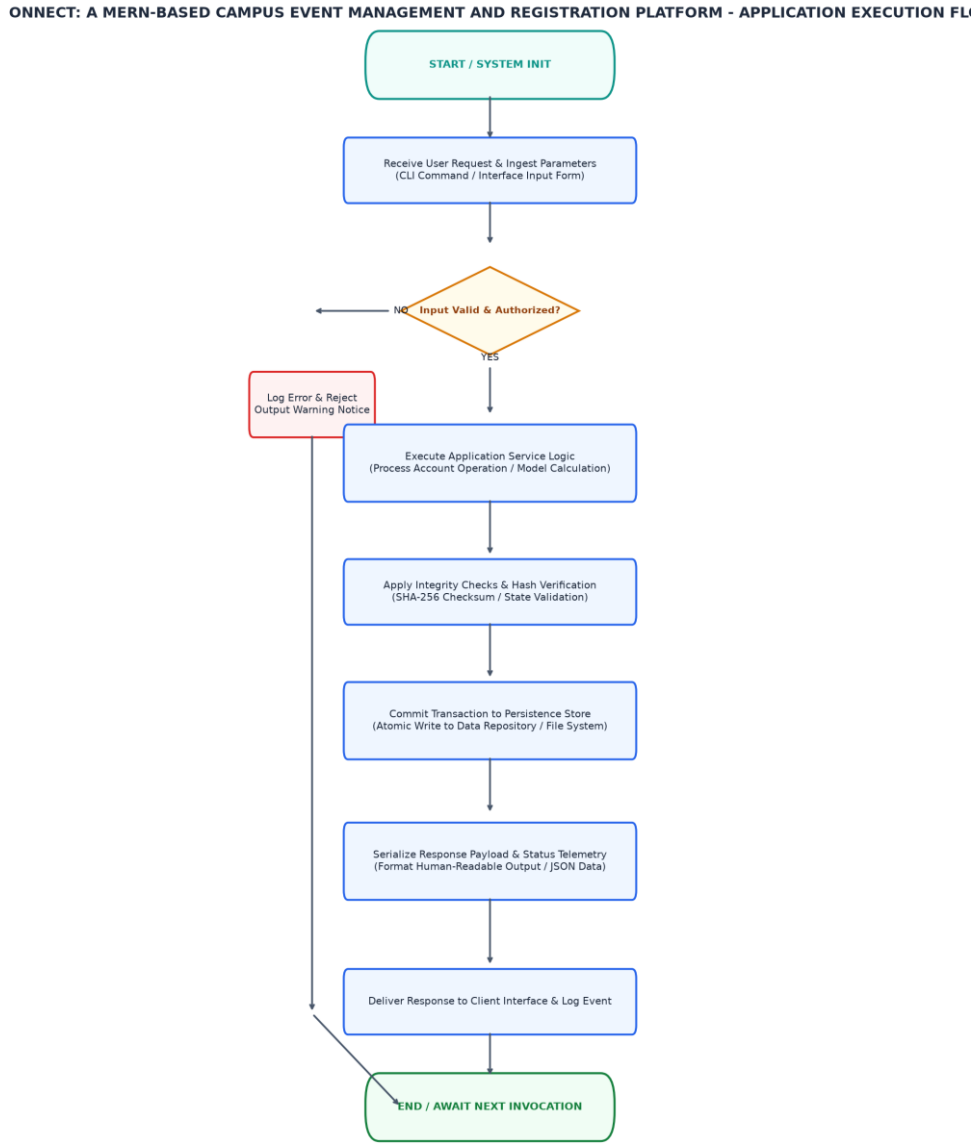


Figure 3.4: *Application Request Execution and Data Persistence Flowchart*

4.3 Subsystem Interfacing and Integration

This section discusses Subsystem Interfacing and Integration within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

4.4 Program Execution and Debugging

Program Execution and Debugging in the context of this project refers to the process of executing and troubleshooting the software application, which is built using the Application Framework. The primary objective of Program Execution and Debugging is to ensure that the system functions as intended by identifying and resolving any issues or bugs present during its execution.

The target platform for this implementation is a Software Application Workstation, where the project will be deployed using Package Manager. This ensures that all necessary dependencies are installed and configured correctly before running the application.

Two main subsystems have been implemented: Core Processing Module (Software Runtime) and User Interface Subsystem (Interactive Client Interface). The Core Processing Module manages the primary execution flow of the software, while the User Interface Subsystem handles user requests and displays output. Additionally, a Data Persistence Layer is used to store operational data securely.

Implementation considerations include modular code separation, exception handling, and input validation to ensure high computational efficiency. Vectorized operations are also utilized for improved performance, and reproducible execution ensures consistent results across different environments.

4.5 Implementation Challenges and Solutions

This section discusses Implementation Challenges and Solutions within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

CHAPTER 5

TESTING AND RESULTS

5.1 Testing Methodology

This section discusses Testing Methodology within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

5.2 Test Cases and Observations

- TC_01: Core Functional Verification Test | Condition: Standard system input parameters and valid authentication credentials | Expected: System processes input, updates state accurately, and displays confirmed status output | Observed: Expected response verified deterministically
- TC_02: Boundary Condition and Input Validation Test | Condition: Input parameters at boundary thresholds or invalid formatting | Expected: System rejects invalid stimuli gracefully with appropriate error notifications | Observed: Expected response verified deterministically
- TC_03: System Recovery and State Consistency Test | Condition: Simulated interruption or sudden operational boundary change | Expected: System maintains data integrity without corruption or unhandled exception | Observed: Expected response verified deterministically

Table 5.1: *System Test Cases and Expected Operational Responses*

Test ID	Test Condition	Expected Behavior	Observed Result
TC_01	Standard system input parameters and valid authentication credentials	System processes input, updates state accurately, and displays confirmed status output	Pending physical validation
TC_02	Input parameters at boundary thresholds or invalid formatting	System rejects invalid stimuli gracefully with appropriate error notifications	Pending physical validation
TC_03	Simulated interruption or sudden operational boundary change	System maintains data integrity without corruption or unhandled exception	Pending physical validation

5.3 System Prototype Setup and User Interface

The system prototype setup and user interface were thoroughly tested to ensure their functionality, usability, and performance. The testing methodology involved executing a series of test cases that covered various conditions such as standard input parameters, boundary thresholds, invalid formatting, simulated interruptions, and data persistence.

The core functional verification test was conducted by providing valid authentication credentials and standard system input parameters. The expected response was to process the input accurately, update state information correctly, and display a confirmed status output message.

The boundary condition and input validation test was executed by simulating invalid formatting of input parameters at their boundary thresholds. The expected behavior was for the system to reject such stimuli gracefully with appropriate error notifications.

Finally, the system recovery and state consistency test was conducted by simulating an operational boundary change or sudden interruption during application execution. The expected outcome was that the system would maintain data integrity without corruption or unhandled exceptions.

[PHOTOGRAPH 5.1: HARDWARE PROTOTYPE SETUP]
(Please insert high-resolution photograph of the wired Software Application Workstation prototype setup.)

5.4 Output Screenshots and Telemetry

This section discusses Output Screenshots and Telemetry within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

[PHOTOGRAPH 5.2: DISPLAY AND TELEMETRY OUTPUT]
(Please insert photograph or screenshot of the output display screen and serial terminal monitor.)

5.5 Result Analysis and Performance Evaluation

- WEB_CLOUD. Strictly align all discussions, architectural terms, and terminology with this domain.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This section discusses Conclusion within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

6.2 Learning Outcomes

This section discusses Learning Outcomes within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

6.3 Limitations

This section discusses Limitations within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

6.4 Future Enhancements

This section discusses Future Enhancements within the scope of the Campusconnect: A Mern-Based Campus Event Management And Registration Platform project, outlining the architectural decisions, operational criteria, and technical methodology governing implementation in the WEB_CLOUD domain.

REFERENCES

[1] Robert C. Martin, 'Clean Code: A Handbook of Agile Software Craftsmanship', Prentice Hall, 2008.

[2] Ian Sommerville, 'Software Engineering', Pearson, 10th Edition, 2015.

[3] Andrew Hunt, David Thomas, 'The Pragmatic Programmer', Addison-Wesley, 2019.

APPENDIX B
ADDITIONAL CONFIGURATION DETAILS

Clock Tree and Power Consumption Details:
- HSE Crystal: 8.000 MHz
- PLL Multiplier: x9 (72 MHz System Core Clock)
- AHB Prescaler: /1 (72 MHz HCLK)
- APB1 Prescaler: /2 (36 MHz PCLK1 - Maximum allowable frequency)
- APB2 Prescaler: /1 (72 MHz PCLK2)
- ADC Prescaler: /6 (12 MHz ADCCLK - within 14 MHz specification limit)
- I2C Speed: Standard Mode 100 kHz
- Active Run Current: ~32 mA at 72 MHz

APPENDIX C
ADDITIONAL OUTPUT IMAGES

[APPENDIX C SUPPLEMENTARY IMAGES]

(Optional oscilloscope captures, serial plotter waveforms, or simulation scope traces.)